



UNIVERSITY  
OF HULL

# Thread pools

# Thread pools

- An alternative approach for deploying threads is to use a thread pool.
- Worker threads in the pool repeatedly retrieve and execute tasks from a task queue.
- Architecturally the approach can produced a very clean design, but it relies on a thread-safe queue.

# Thread pool

```
1.  def worker(threadId, taskQueue):
2.      safePrint('Starting worker', threadId)

3.      task = taskQueue.get()
4.      while task is not None:

5.          safePrint ('Running task',task,'on worker',threadId)
6.          sleep(random())                #simulate work
7.          taskQueue.task_done()
8.          task = taskQueue.get()

9.      taskQueue.task_done()

10. safePrint('Ending worker',threadId)
```

# Thread pool

```
1.  safePrint('Starting main programme')

2.  taskQueue = Queue(3)
3.  threadIds = range(2)
4.  for i in threadIds:
5.      threading.Thread(target = worker, args = [i, taskQueue]).start()

6.  taskIds = range(8)
7.  for i in taskIds:
8.      taskQueue.put(i)      #safePrint('Queued task', i)

9.  for i in threadIds:
10.     taskQueue.put(None)

11. taskQueue.join()  # wait for all work to finish

12. safePrint('Ending main programme')
```

# Thread pool

```
Starting main programme
Starting worker 0
Starting worker 1
Running task 0 on worker 0
Running task 1 on worker 1
Running task 2 on worker 1
Running task 3 on worker 1
Running task 4 on worker 0
Running task 5 on worker 0
Running task 6 on worker 1
Running task 7 on worker 0
Ending worker 1
Ending worker 0
Ending main programme
```

```
Starting main programme
Starting worker 0
Starting worker 1
Queued task 0
Running task 0 on worker 0
Queued task 1
Running task 1 on worker 1
Queued task 2
Queued task 3
Queued task 4
Running task 2 on worker 1
Queued task 5
Running task 3 on worker 0
Queued task 6
Running task 4 on worker 1
Queued task 7
Running task 5 on worker 0
Running task 6 on worker 1
Running task 7 on worker 0
Ending worker 0
Ending worker 1
Ending main programme
```

# Thread pool, the queue

- What synchronisation is required within the Queue data structure?
- How would you implement `queue.task_done()` ?
- How would you implement `queue.join()` ?



UNIVERSITY  
OF HULL

# Hints and tips for threading

# When to use

- When there is a need to schedule work in the background and still update the foreground.
  - E.g. loading a terrain database in the background and rendering in the foreground
- Look for a logical division between threads
- Avoid significant dependencies between threads
  - Results in excessive suspension
- Aim for independent execution
- Two good architectures
  - Always running and independent of first thread. Makes data available to first thread
  - Almost never runs, sleeps, waiting for an event. When the event occurs it runs quickly and then sleeps again



# Warning

- Never underestimate the complexity of creating a multi-threaded application
- Work through all possible combinations of data access
- Design to avoid deadlocks, livelocks and race conditions
- Consider data integrity

# More advanced TCP/IP

# Comparison of TCP vs UDP

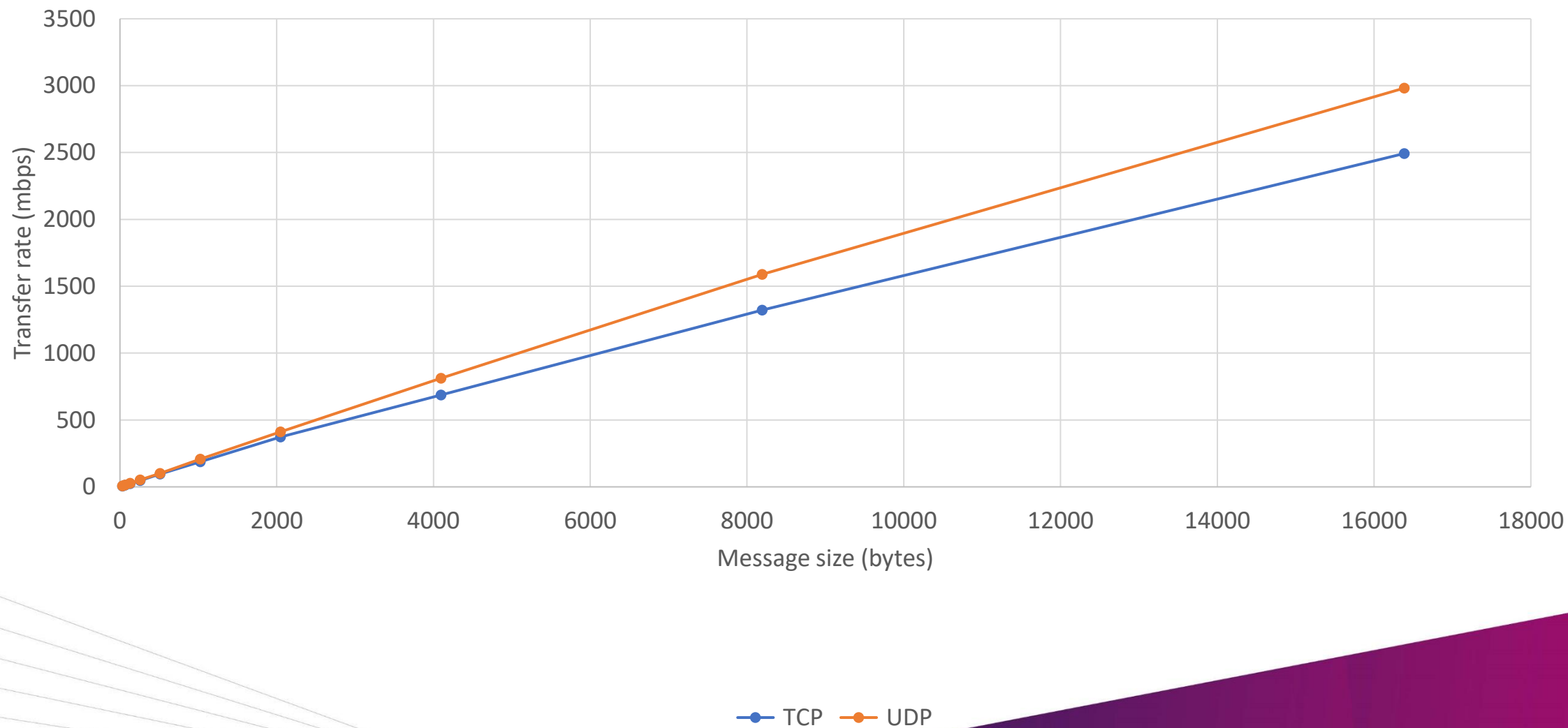
- Reliability

- TCP– data always arrives
  - Ensures lost, corrupted, duplicate and fragmented packets are handled correctly
- UDP– unreliable
  - Lost, corrupted and fragmented packets are silently dropped
  - Duplicate data is passed to the application

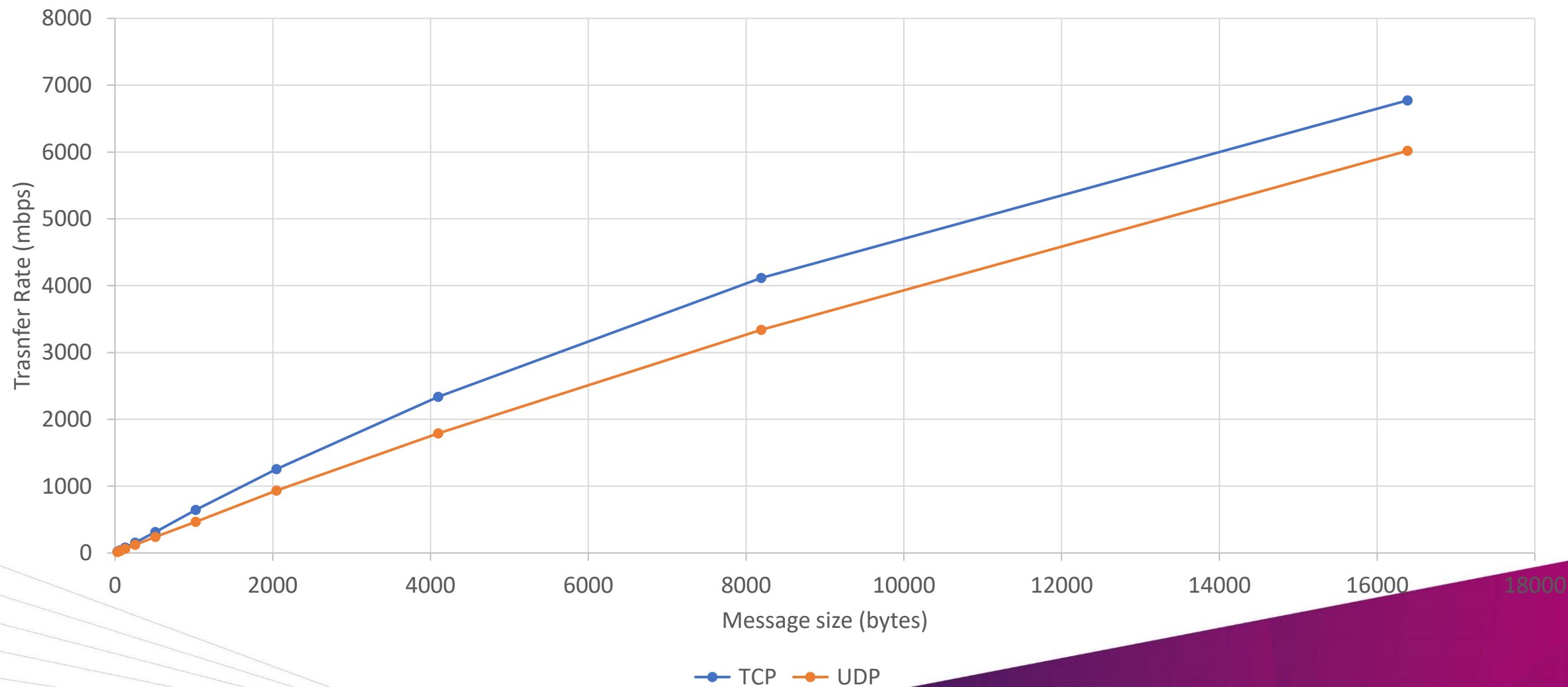
- Performance

- TCP – bundles data for efficiency
- UDP – less latency as code is simpler

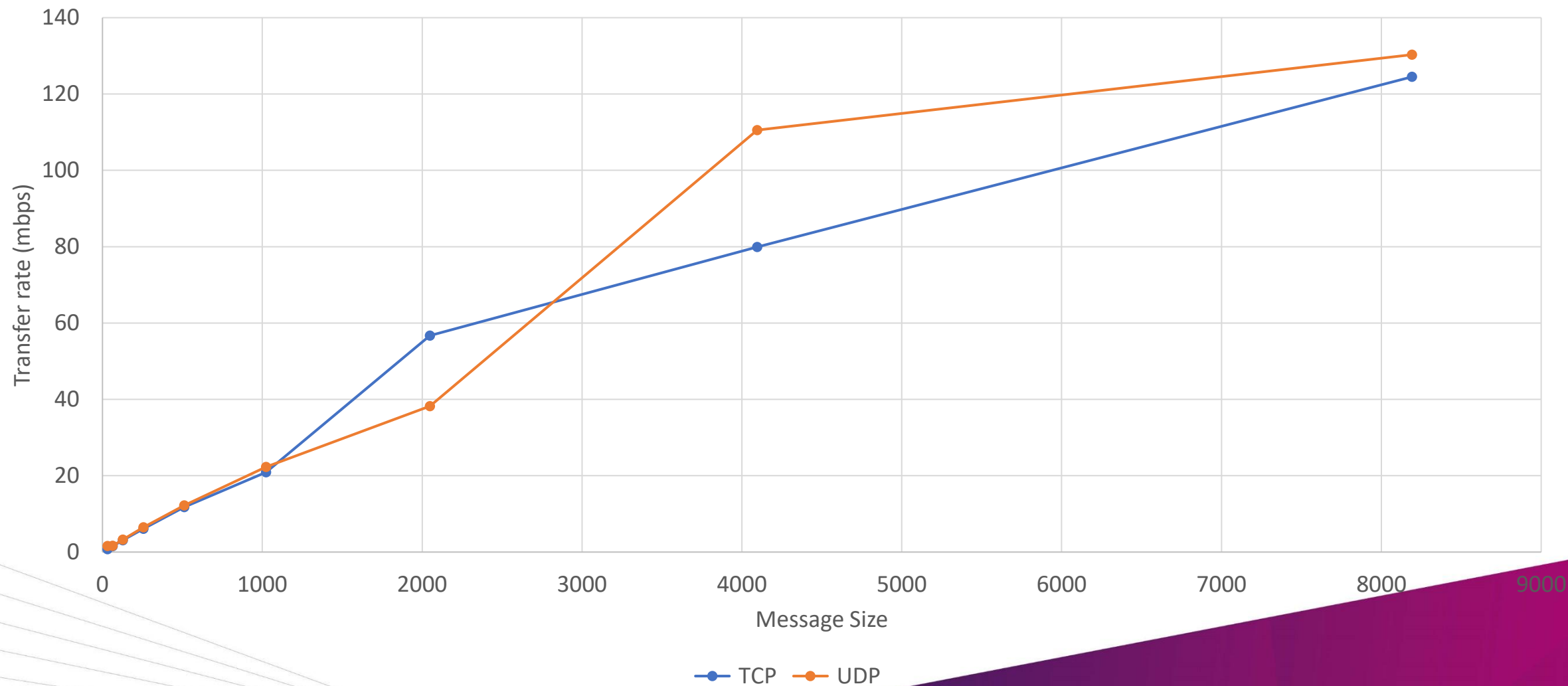
# Local host (127.0.0.1)



# Local host (127.0.0.1)



# Local subnet



# Example combining UDP and TCP

- Use UDP for the initialization via a 'broadcast' and TCP for the data transfer.
  - Welcome server – UDP
  - Echo server - TCP

# UDP/TCP server

```
1.  def welcomeServer():
2.      port = 4200
3.      ip = '' #INADDR_ANY
4.      addr = (ip, port)

5.      print ('\nOpening welcome server')
6.      dataSocket = socket(AF_INET, SOCK_DGRAM)
7.      dataSocket.bind(addr)

8.      message, peerAddr = dataSocket.recvfrom(bufferSize)
9.      print ('Echoing message ', message, ' to ', peerAddr)
10.     dataSocket.sendto(message, peerAddr)

11.     print ('Closing welcome server')
12.     dataSocket.close()

13. def main():
14.     message = ''
15.     while (message != b'q'):
16.         welcomeServer()
17.         message = echoServer()
```





# UDP/TCP server

```
1.  def echoServer():
2.      port = 4201
3.      ip = '' #INADDR_ANY
4.      addr = (ip, port)

5.      print ('\nOpening echo server')
6.      listeningSocket = socket(AF_INET, SOCK_STREAM)
7.      listeningSocket.bind(addr)
8.      listeningSocket.listen(5)

9.      print ('Listening on ', addr)
10.     dataSocket, peerAddr = listeningSocket.accept()
11.     print ('Connecting ', dataSocket.getsockname(), ' to ', peerAddr)

12.     message = dataSocket.recv(bufferSize)
13.     print ('Echoing message ', message)
14.     dataSocket.send(message)
15.     dataSocket.close()

16.     print ('Closing echo server')
17.     listeningSocket.close()
18.     return message
```

# UDP/TCP client

```
1.  port = 4200
2.  ip = '<broadcast>'
3.  addr = (ip, port)
4.  bufferSize = 1000
5.  message = input('Enter message (def=Hello World, quit=q):').encode()
6.  if not message:
7.      message = b'Hello World'

8.  print ('\nOpening UDP socket')
9.  udpSocket = socket(AF_INET, SOCK_DGRAM)
10. udpSocket.setsockopt(SOL_SOCKET, SO_BROADCAST, 1)

11. udpSocket.sendto(b'It is me!!!', addr)
12. welcomeMessage, serverAddr = udpSocket.recvfrom(bufferSize)
13. print ('Receiving data ', welcomeMessage, ' from ', serverAddr)

14. print ('Closing UDP socket')
15. udpSocket.close()
```

# UDP/TCP client, contd

```
16. print ('\nOpening TCP socket')
17. tcpSocket = socket(AF_INET, SOCK_STREAM)
18. serverAddr = (serverAddr[0], serverAddr[1]+1)
19. tcpSocket.connect(serverAddr)
20. print ('Connecting ', tcpSocket.getsockname(), ' to ', tcpSocket.getpeername())

21. tcpSocket.send(message)
22. message = tcpSocket.recv(bufferSize)
23. print ('Receiving data ', message)

24. print ('Closing TCP socket')
25. tcpSocket.close()
```

# Port numbers

- 1-1023: Reserved Ports, for standard protocols
- 1024-5000: Registered Ports
  - Operating systems use for client ports
- 5001-49151: Registered Ports
  - Check registered list on <http://www.iana.org/assignments/port-numbers>
- 49152-65535: Dynamic Ports
  - Operating systems use these when random ports are needed

# Winsocks and threading

- The Winsock standard does not explicitly require thread safety.
- Considered good practice to have only one thread access a given socket.
- If multiple threads access a single socket, only one thread will receive the data.
- If multiple threads wait for an event on a single socket, only one thread will be notified of the event.

# Winsock communication conventions

- Blocking sockets
  - Default behaviour
  - Does not return until completion or error
- Non-blocking sockets
  - Returns immediately, even with no data
  - Sockets need to be polled to determine presence of data
- Asynchronous sockets
  - Non blocking, so returns immediately, even with no data
  - Presence of data, triggers an event
- Select()
  - Non blocking, but suspends thread
  - Presence of data, pulses thread
- Event objects
  - Similar to select, but more efficient
- Overlapped I/O
  - Uses win32's I/O mechanism, more efficient than others

# Timers for Blocking functions

- Blocking functions do not return until a condition has been met.
  - `send()`, `recv()`, `connect()`
- Blocking functions can optionally timeout and return
- Length of timeout can be set using `setsockopt()`
  - `SO_SNDTIMEO` and `SO_RCVTIMEO` for `send()` and `recv()` timeouts
- Note: There is no timeout options for `accept()`
  - Closing the socket from another thread will unblock an `accept()`

# Asynchronous sockets

```
1. // Create listening socket
2. s = socket(AF_INET, SOCK_STREAM, 0);
3. bind(s, (sockaddr *)&peer, sizeof(peer));

4. // Request a windows message if the socket generates FD_ACCEPT/FD_CLOSE events
5. WSAAsyncSelect(s, hWnd, wMsg, FD_ACCEPT|FD_CLOSE);

6. if (listen(s, 5)==SOCKET_ERROR)
7.     cout << "Listening failed" << endl;
8. cout << "Listening" << endl;
```



# Select()

```
1. fd_set allMasks;
2. FD_ZERO (&allMasks);
3. FD_SET (server1.handle(), &allMasks);
4. FD_SET (server2.handle(), &allMasks);

5. SocketStream newStream;
6. fd_set readMasks = allMasks;

7. if (select(0, &readMasks, NULL, NULL, NULL) <= 0) // blocks
8.     cout << "Server port error" << endl;
9. if (FD_ISSET (server1.handle(), &readMasks))
10.     server1.accept(newStream);
11. else if (FD_ISSET (server2.handle(), &readMasks))
12.     server2.accept(newStream);
```

# Event sockets

```
1. // Create listening socket
2. s = socket(AF_INET, SOCK_STREAM, 0);
3. bind(s, (sockaddr *)&peer, sizeof(peer));

4. // Create new event
5. WSAEVENT e = WSACreateEvent();

6. // Requests a new event is triggered if the socket generates
7. // FD_ACCEPT /FD_CLOSE events
8. WSASyncSelect(s, e, FD_ACCEPT|FD_CLOSE );

9. if (listen(s, 5)==SOCKET_ERROR)
10.     cout << "Listening failed" << endl;
11.cout << "Listening" << endl;
```

# Event Sockets

- `WSAAsyncSelect(s, e, RD_READ );`
  - Requests windows messaging if and when data is ready to be read.
  - If not all the data in the socket's buffer is read, a new event is generated.

# Timers for Non-blocking functions

- Non blocking sockets with `select()`
  - `Select` will suspend the thread
- Asynchronous sockets
  - `SetTimer()`
- Event objects
  - `WSAWaitForMultipleEvents()`
- Waitable Timers
  - `CreateWaitableTimers()`, `WaitForMultipleObjects()`

# Considerations

- Blocking sockets (no threading)
  - Limited use, as application freezes
  - Only works for a single connection
- Blocking sockets with threading
  - Efficient because OS does scheduling
  - Provided very low number of connections (<10)
  - Significant context switching when number of connections get large
  - Usually requires synchronization between threads which is non trivial to implement (critical section etc)
  - Avoid if you are not already using threads in the application
- Non-Blocking sockets
  - Only use with low number of connections (<100)
  - Inefficient due to the need to poll for data
  - More design options available than with blocking sockets
  - Does not require threading

# Considerations, contd.

- Asynchronous sockets
  - Good with low volume of data and low number of connections (<100)
  - Integrates well with GUI because message loop already exists
  - A server would require a dummy GUI with a message loop
  - Does not require threading
- Select()
  - Use only with low number of connections (<10)
  - Inefficient and does not scale well
  - Requires threading
  - Compatible with most systems
- Events
  - Good with low volume of data and low number of connections (<100)
  - Limited to 64 connections\*
  - Requires threading
  - Does not require a message loop
- Overlapped I/O
  - Only option for high performance servers (>100 connections)
  - Difficult to implement

# Peeking

- Peeking is looking ahead into the TCP buffer
- `recv()` with a `MSG_PEEK` parameter
- Avoid
  - Ref. Microsoft's Knowledge Base Q192599

# Nagle algorithm

- Windows TCP/IP implements the Nagle algorithm
  - <http://www.ietf.org/rfc/rfc896.txt>
- The Nagle algorithm coalesces small packets into larger packets to reduce network overhead.
- If the size of a packets is less than the MTU and there are no waiting packets, then the packet is sent immediately
- If the size of a packets is less than the MTU and there are waiting packets, then the packet is coalesced with the waiting packets.
- Nagle algorithm is not used in loopback networking
- Nagle algorithm can be disabled.



# Avoid the following

- `Select()`
- Single byte send and receives
- Repeated calls to `WSAStartup()` and `WSACleanup()`
- Repeated setup and closure of sockets
- Binding to a port with a client application
- `MSG_PEEK`
- Large UDP messages

# Avoid the following assumptions

- UDP reliability
- TCP messages are received in full
- TCP receive buffers larger than the message size